



Universität Stuttgart

Institut für Parallele und Verteilte Systeme
Anwendersoftware

HE und CIA: Schild und Schwert der Datensicherheit

Seminararbeit

Vertiefungsthemen Data Science (SS 2023)

Betreuer: Daniel Del Gaudio

Maximilian Friedl

Stuttgart, 25.08.2023

HE und CIA: Schild und Schwert der Datensicherheit

Maximilian Friedl

st178725@stud-uni-stuttgart.de

Keywords: Homomorphe Verschlüsselung, Kryptographie, Datensicherheit, CIA - Triade, Maschinelles Lernen, Paillier Crypto System

Zusammenfassung Homomorphe Verschlüsselung (HE) erlaubt Berechnungen auf verschlüsselten Daten, insbesondere in Cloud-Umgebungen, Finanzwesen aber auch in der Netzwerküberwachung.

Es gibt drei Varianten: partielle (PHE), semihomomorphe (SWHE) und vollständige homomorphe Verschlüsselung (FHE), wobei FHE das ultimative Ziel darstellt. FHE wurde von Craig Gentry eingeführt und erlaubt unbegrenzte Berechnungen, ohne dass die Daten entschlüsselt werden müssen. Dadurch werden die Daten während des Verarbeitungsprozesses geschützt [9].

Die Technologie erfüllt alle Aspekte der CIA-Triade (Vertraulichkeit, Integrität, Verfügbarkeit), ist jedoch in der Praxis noch komplex und mit Herausforderungen verbunden [17]. Aktuelle Forschungen zielen darauf ab, die Leistung von Homomorpher Verschlüsselung (HE) zu verbessern. Zudem findet HE erste Anwendungen im maschinellen Lernen, wo sie den Schutz der Privatsphäre und den sicheren Datenaustausch ermöglicht [29].

1 Einleitung

In einer Welt, in der Daten das neue Öl, das neue Gold sind, ist die Sicherheit dieser Daten von größter Bedeutung [12]. Homomorphe Verschlüsselung (HE) bietet eine vielversprechende Lösung für dieses Problem, indem sie Berechnungen auf verschlüsselten Daten ermöglicht, ohne diese zu entschlüsseln [1]. In allen bisherigen Verschlüsselungssystemen müssen Schlüssel mit Zweiten geteilt werden. Dabei kann es sich um öffentliche oder private Schlüssel handeln. Je nach genutztem Dienst liegen die Daten auf einem Server, in der Cloud des Anbieters oder bei anderen, unbekannten Dritten. Diese sollten im Sinne des Datenschutzes grundsätzlich keinen Zugriff auf diese Daten haben. Mit Hilfe von HE kann der Zugriff auf diese Daten unterbunden werden, die Verarbeitung der Daten jedoch aufrechterhalten werden. Dies reicht von einfachen Programmen, die z.B. Daten akkumulieren oder auswerten, bis hin zu maschinellen Lernen, mit dem ganze Datensätze von Banken überprüft werden können [13].

Damit der Verlauf einer Verschlüsselung bis hin zu einer Abfrage und deren Ergebnis deutlicher wird, ist in Abbildung 1. ein abstrahiertes Beispiel zu sehen.

1. Ein*e Nutzer*in verschlüsselt seine/ihre Daten mit einem *Encryption Key*.
2. Dies verschlüsselten Daten werden an den Server gesendet.
3. Der/die Nutzer*in stellt eine Anfrage an den Server, wie zum Beispiel eine SQL-Query.
4. Der Server kann die Daten die dafür benötigt werden, ohne zu entschlüsseln, abfragen und verarbeiten.
5. Das Ergebnis wird dann wieder an den Nutzer gesendet.
6. Der/die Nutzer*in entschlüsselt die Daten mit seinem *Decryption Key* und kann das Ergebnis einsehen.

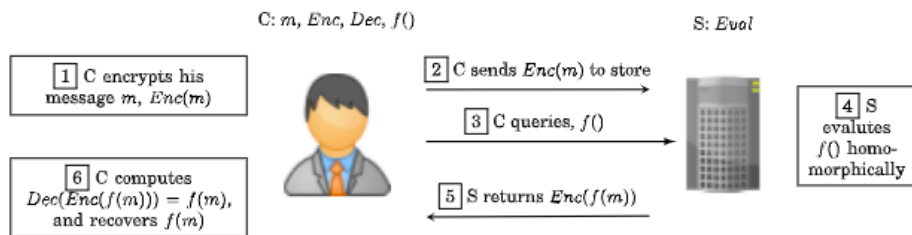


Abbildung 1. Beispiel für ein HE Szenario, in dem ein Client Anfragen an einen Server stellt [1].

Homomorphe Verschlüsselung (HE), die ihren Namen vom Konzept des Homomorphismus ableitet, ist eine spezielle Form der Verschlüsselung, die es ermöglicht, Berechnungen auf verschlüsselten Daten durchzuführen, ohne dass eine vorherige Entschlüsselung erforderlich ist. Die Forschung in der Kryptographie hat zum Ziel, die Datensicherheit zu erhöhen. Samonias und Coss haben in Ihrem Artikel „The CIA strikes back: redefining confidentiality, integrity and availability in security“ [21] die wichtigsten Punkte erörtert. Dies wird im nächsten Abschnitt beleuchtet.

Im Anschluss werden die Varianten der HE, welche maßgebend für die Sicherheit der Daten sind, betrachtet. Jede der drei Varianten; *SWHE: Somewhat HE*, *PHE: Partially HE*, *FHE: Fully HE* hat andere Stärken und Schwächen.

Nachdem die Begrifflichkeiten und das Konzept der HE erklärt sind, widmen wir uns den mathematischen Hintergründen. Die Homomorphie, die in der abstrakten Algebra eine Abbildung bezeichnet, erhält die algebraische Struktur zwischen zwei algebraischen Mengen. Eine Abbildung ist in diesem Zusammenhang eine Funktion oder Operation, die Eingaben aus dem Definitionsbereich annimmt und ein Element aus dem Wertebereich ausgibt. Der Homomorphismus erhält also die algebraische Struktur der Operation, z. B. Addition oder Multiplikation, zwischen den beiden Mengen.

Desweiteren verwendet die **FHE** sogenannte *Ideale Gitter*, das *Ring-LWE-Problem*

und *Bootstrapping*. Hier wird die Arbeit von Craig Gentry, welcher als Erfinder der FHE gilt, betrachtet.

Homomorph verschlüsselte Daten bieten eine einzigartige Möglichkeit für das maschinelle Lernen (ML). Die Daten können in ihrem verschlüsselten Zustand verarbeitet werden, ohne dass diese jemals in den ursprünglichen Klartext entschlüsselt werden müssen.

In den Bereichen der Medizinischen Informatik sowie der Bioinformatik, wurden hier bereits erste Erfolge erzielt [29]. FHE könnte es ermöglichen ML auf verschlüsselten Daten durchzuführen, ohne einen Kompromiss zwischen Genauigkeit und Geheimhaltung einzugehen [26]. Es werden die Vorteile und Herausforderungen der Anwendung von HE in ML untersucht und praktische Beispiele und Fallstudien vorgestellt [13, 18].

Im darauf folgenden Abschnitt werden die Anwendungsfälle der HE, sowie deren Vorteile betrachtet. Da die HE noch relativ „jung“ ist und die Implementierung sehr komplex ist, sind diese Anwendungsfälle größtenteils theoretischer Natur.

Abschließend werden zukünftige Möglichkeiten der HE, sowohl in der Praxis als auch in der Forschung aufgezeigt.

1.1 Ziele meiner Arbeit

Mein Ziel ist es, das Wissen über die CIA-Triade zu verbreiten und ihre Bedeutung für die Datensicherheit, insbesondere die grundlegenden Anforderungen an die Datensicherheit, zu verdeutlichen.

Außerdem analysiere ich detailliert die verschiedenen Varianten der homomorphen Verschlüsselung: PHE, SWHE und FHE, indem ich die mathematischen Grundlagen erkläre. Damit erweitere ich das Wissen über die Funktionsweise und die Einsatzmöglichkeiten dieser Verschlüsselungsverfahren.

Ein weiterer Schwerpunkt meiner Arbeit ist der Zusammenhang zwischen homomorpher Verschlüsselung und maschinellem Lernen. Ich zeige, wie Daten in verschlüsselter Form verarbeitet und analysiert werden können, ohne sie vorher entschlüsseln zu müssen. Dies eröffnet neue Perspektiven für den Schutz der Privatsphäre und den sicheren Austausch sensibler Informationen im Kontext des maschinellen Lernens.

Darüber hinaus stelle ich konkrete Anwendungsfälle der homomorphen Verschlüsselung in der Medizin vor.

Insgesamt trage ich mit meiner wissenschaftlichen Arbeit dazu bei, das Bewusstsein für homomorphe Verschlüsselung zu schärfen und mögliche neue Forschungsrichtungen in diesem aufstrebenden Forschungsgebiet aufzuzeigen. Mein Ziel ist es, dem Leser ein tieferes Verständnis der verschiedenen Aspekte der Datensicherheit, der homomorphen Verschlüsselung und ihrer Anwendung im Kontext des maschinellen Lernens zu vermitteln.

2 Vom Schild zum Schwert: Grundlagen und Begrifflichkeiten

Um ein besseres Verständnis der homomorphen Verschlüsselung (HE) zu erhalten, werden zunächst einige zentrale Begriffe erläutert. Bevor wir uns den spezifischen Varianten der homomorphen Verschlüsselung (HE) widmen, nehmen wir uns Zeit, um das grundlegende Prinzip der Datensicherheit zu betrachten – einem zentralen Pfeiler in der Welt der Kryptographie. Dieses Konzept ist von zentraler Bedeutung, um die Tragweite der HE zu erfassen.

Im Anschluss werden die drei unterschiedlichen HE-Varianten erörtert. Die Zusammenführung dieses Wissens erfolgt in Abschnitt 3, während Abschnitt 4 die praktischen Anwendungen und Vorteile dieser Technologie beleuchtet.

Ein zentrales Ziel der kryptographischen Forschung ist die Erhöhung der Datensicherheit. Im Hinblick auf die klassische CIA-Triade haben Samonas und Coss in ihrem Artikel "THE CIA STRIKES BACK: REDEFINING CONFIDENTIALITY, INTEGRITY AND AVAILABILITY IN SECURITY" [21] eine gründliche Analyse und Neubewertung der CIA-Triade, bestehend aus Vertraulichkeit, Integrität und Verfügbarkeit, vorgelegt. Diese Erkenntnisse bilden den Ausgangspunkt für meine Untersuchungen zur Datensicherheit homomorpher Verschlüsselung.

2.1 Schneidende Klarheit: Dem Schwert der CIA-Triade auf der Spur

Informationen sind das Herzstück von Unternehmen und Behörden. Maturdi et al. [17] haben sich in Ihrem Paper auf die Datensicherheit von Big Data fokussiert. Big Data, seien es nun wirklich so große und komplexe Datenmengen, die spezielle Verarbeitungsmethoden erfordern, da sie die Kapazität herkömmlicher SQL-Datenbanken übersteigen und nicht das Umherwerfen von Buzzwords.¹ Sie kommen zu dem Entschluss, dass Big Data zunehmend in Forschung und Wirtschaft eingesetzt wird, was erhebliche Vorteile mit sich bringt, aber auch Herausforderungen in Bezug auf Datensicherheit und Datenschutz aufwirft.

Die Menge der von Menschen erzeugten Daten hat in den letzten Jahren stark zugenommen, was zu einer explosionsartigen Zunahme von Big Data geführt hat. Während 2010 noch 2 Zettabyte an digitalen Daten generiert wurden, sind es 2022 bereits 103 Zettabyte. 2027 könnte dieser Wert bereits bei 280 Zettabyte liegen [24]. Diese Daten stammen aus verschiedenen Quellen wie der wissenschaftlichen Forschung, der Finanz- und Wirtschaftsinformatik, der Verwaltung, der Internetsuche, sozialen Netzwerken und anderen. Der Wert von Big Data ist beträchtlich, und viele Organisationen auf der ganzen Welt sammeln und analysieren ihre Geschäftsprozessdaten, um ihre interne Entscheidungsfindung zu verbessern. Big Data treibt die Wirtschaft und die wissenschaftliche Forschung

¹ <https://www.computerwoche.de/a/von-big-data-reden-aber-small-data-meinen,3068465>

voran, verändert traditionelle Geschäftsmodelle und wissenschaftliche Methoden und schafft neue Möglichkeiten durch Datenanalyse.

Die Erforschung und Nutzung des außerordentlichen Werts von Big Data geht jedoch mit erhöhten Risiken für die Sicherheit und den Schutz der Privatsphäre einher. Unternehmen wie Amazon, Google und Facebook sammeln und analysieren unsere persönlichen Daten, was Bedenken hinsichtlich der Sicherheit und des Schutzes der Privatsphäre aufwirft. Big Data Security bezieht sich im Allgemeinen auf die Nutzung von Big Data, um Lösungen zu implementieren, die die Sicherheit, Zuverlässigkeit und den Schutz eines verteilten Systems erhöhen. Es gibt verschiedene Methoden und Techniken, um Sicherheit und Datenschutz bei Big Data zu gewährleisten, darunter Passwörter, kontrollierter Zugang, Zwei-Faktor-Authentifizierung, Kryptografie und mehr.

Zusammenfassend lässt sich sagen, dass Big Data erhebliche Vorteile bietet, aber auch Herausforderungen in Bezug auf Sicherheit und Datenschutz mit sich bringt. Es ist wichtig, geeignete Methoden und Techniken zu implementieren, um die Sicherheit und den Datenschutz bei der Nutzung von Big Data zu gewährleisten. Ihr Verlust kann nicht nur rechtliche Konsequenzen und Imageschäden nach sich ziehen, sondern auch den Geschäftsbetrieb empfindlich stören. Zahlreiche Vorfälle im Bereich der Informationssicherheit, wie der Verlust von Nutzerdaten, gezielte Angriffe auf Unternehmen oder Krankenhäuser, unterstreichen die Bedeutung dieses Themas [28]. Im Falle von Krankenhäusern wurden bereits Patientendaten und Server komplett verschlüsselt. Dies stellt eine erhebliche Störung der Betriebsabläufe dar und hat bereits zum Tod einer Patientin geführt². Die Vergangenheit hat gezeigt, dass das Sicherheitsniveau oft unzureichend und nicht angemessen ist. Nach Vorfällen wie diesem steht es außer Frage, dass die Informationstechnologie entsprechend geschützt werden muss.

Während die IT-Systeme oft mit guten Schutzmaßnahmen ausgestattet sind, sind Daten oder eben Informationen oft nur schwer zu greifen. Bei diesen muss die Datensicherheit über alle Geschäftsprozesse vorhanden sein. Sollte in einer Phase dieses Prozesses die Datensicherheit nicht gewährleistet sein, so ist die Datensicherheit im Ganzen nicht gegeben [28].

Daher gibt es die CIA-Triade [21]:

- **Confidentiality (Vertraulichkeit)**
bezieht sich auf den Schutz von Informationen vor unbefugtem Zugriff oder Offenlegung. Vertrauliche Informationen sollen nur autorisierten Personen oder Systemen zugänglich sein.
- **Integrity (Integrität)**
bezieht sich auf die Gewährleistung der Richtigkeit, Vollständigkeit und Unveränderlichkeit von Informationen. Es bedeutet, dass Informationen vor

² <https://www.handelsblatt.com/technik/sicherheit-im-netz/cyberkriminalitaet-todesfall-nach-hackerangriff-auf-uni-klinik-duesseldorf/26198688.html>

unbefugter Manipulation oder unautorisierten Änderungen geschützt sein sollten.

– **Availability (Verfügbarkeit)**

bezieht sich auf die Gewährleistung, dass Informationen und Systeme jederzeit und ohne Unterbrechung für autorisierte Benutzer zugänglich sind. Informationen sollten vor Ausfällen, Störungen oder anderen Bedrohungen geschützt sein, die ihre Verfügbarkeit beeinträchtigen könnten.

Homomorphe Verschlüsselung kann die CIA-Triade beim Datenschutz unterstützen. Sie ermöglicht es, verschlüsselte Daten zu verarbeiten, ohne sie vorher entschlüsseln zu müssen. Dies hat die folgenden Auswirkungen auf die verschiedenen Aspekte der CIA-Triade:

– **Vertraulichkeit:**

Homomorphe Verschlüsselung gewährleistet Vertraulichkeit, indem sie es ermöglicht, Daten in verschlüsselter Form zu verarbeiten, ohne die eigentlichen Klartextdaten zu kennen. Dies bedeutet, dass vertrauliche Informationen während der Verarbeitung geschützt bleiben, da sie niemals entschlüsselt werden müssen.

– **Integrität:**

Da die Daten während der Verarbeitung verschlüsselt bleiben, bleibt die Integrität der Informationen erhalten. Es können keine unbefugten Änderungen an den Daten vorgenommen werden, da sie verschlüsselt bleiben und nur autorisierte Operationen auf den verschlüsselten Daten durchgeführt werden können.

– **Verfügbarkeit:**

Die Verfügbarkeit von Daten kann durch homomorphe Verschlüsselung beeinflusst werden. Da die Verarbeitung verschlüsselter Daten in der Regel rechenintensiver ist als die Verarbeitung von Klartextdaten, kann es zu einem gewissen Performanceverlust kommen.

Dies bedeutet, dass die Verfügbarkeit der Daten beeinträchtigt werden kann, wenn die Verarbeitungsgeschwindigkeit reduziert wird.

Die homomorphe Verschlüsselung wird jedoch ständig weiterentwickelt, um die Leistung zu verbessern und die Auswirkungen auf die Verfügbarkeit zu minimieren.

Homomorphe Verschlüsselung bietet somit die Möglichkeit, Datenverarbeitung und Datensicherheit miteinander zu verbinden.

Die homomorphe Verschlüsselung (HE) zielt darauf ab, die Grundsätze der CIA-Triade - *Vertraulichkeit, Integrität und Verfügbarkeit* - zu erfüllen. HE ermöglicht es, Berechnungen wie das Addieren von Werten auf verschlüsselten Daten durchzuführen. Allerdings könnten Fragen zur Integrität auftreten, wenn solche Operationen außerhalb des Verschlüsselungsschemas oder durch nicht autorisierte Prozesse durchgeführt werden. Es ist wichtig zu betonen, dass Änderungen an den verschlüsselten Daten nur von berechtigten Nutzern oder durch Befehle vorgenommen werden können. Es ist jedoch unerlässlich, sich der potenziellen Risiken bewusst zu sein und Mechanismen zur Überprüfung und Aufrechterhaltung der

Datenintegrität einzusetzen.

Obwohl die homomorphe Verschlüsselung vielversprechend erscheint, ist ihre tatsächliche Implementierung ein komplexes Unterfangen, welches in der Praxis noch immer mit technischen Schwierigkeiten und bedeutenden Limitierungen konfrontiert ist.

2.2 SWHE, PHE, FHE: Die verschiedenen Schilde im Arsenal der Datensicherheit

Es gibt verschiedene Arten von homomorpher Verschlüsselung, die sich in ihrer Fähigkeit unterscheiden, verschiedene Arten von Berechnungen auf den verschlüsselten Daten durchzuführen. Dazu gehören die partielle homomorphe Verschlüsselung (PHE), die semihomomorphe Verschlüsselung; auch Somewhat HE genannt (SWHE) und die vollständige homomorphe Verschlüsselung (FHE).

1. PHE ermöglicht es, bestimmte Arten von Berechnungen, wie zum Beispiel Additionen oder Multiplikationen, begrenzt auf verschlüsselten Daten durchzuführen, ohne dass eine Entschlüsselung erforderlich ist. Ein bekanntes Beispiel für ein PHE-Schema ist das RSA-Verschlüsselungsschema, jedoch ist dies nur homomorph für die Multiplikation [1].
2. SWHE, auch leicht homomorphe Verschlüsselung genannt, erweitert die Möglichkeiten der PHE, indem sie eine begrenzte Anzahl von Berechnungen an den verschlüsselten Daten erlaubt, bevor der Rauschpegel zu hoch wird und eine Entschlüsselung erforderlich wird. Diese Methode ist flexibler als PHE, hat aber auch ihre Grenzen.
3. FHE ist das ultimative Ziel der homomorphen Verschlüsselung. Sie ermöglicht eine unbegrenzte Anzahl von Berechnungen auf verschlüsselten Daten, ohne dass eine Entschlüsselung erforderlich ist. Dies bedeutet, dass Daten sicher in einer Cloud-Umgebung gespeichert und verarbeitet werden können, ohne dass die Privatsphäre des Nutzers gefährdet wird.

3 Die Taktik hinter dem Schild: Mathematische Strategien der Homomorphen Verschlüsselung

Nach der Definition der wichtigsten Begriffe können wir uns nun Craig Gentry zuwenden, der oft als Erfinder der homomorphen Verschlüsselung bezeichnet wird.

Craig Gentry's 2009 Paper: Fully Homomorphic Encryption Using Ideal Lattices [9] ist wahrscheinlich die wichtigste wissenschaftliche Arbeit auf diesem Gebiet. Für seine Arbeit erhielt er den ACM Doctoral Dissertation Award for Innovation in Encryption Technology [27].

Gentry hat in seinem Paper als erstes eine Fully Homomorphic Encryption (FHE) dargestellt, während bisher nur PHE und SWHE möglich waren. Seine Arbeit basiert jedoch auch auf einer anderen Arbeit; Rivest, Adleman and Dertouzos':

On data banks and privacy homomorphisms [20] von 1978. Letztere, ohne Dertouzos, erfanden 2002 das RSA-Verschlüsselungsschema und gewannen dafür den ACM A.M. Turing Award [27].

Gentry wird dennoch als „Erfinder“ angesehen, da Rivest et. al. hauptsächlich die Grundlagen für HE gelegt haben. Ihre Arbeit ließ einige Fragen offen. Sie erkannten jedoch bereits, dass Privacy-Homomorphismen einen innovativen Ansatz zur Wahrung der Privatsphäre von zu verarbeitenden Daten bieten, wie FHE bis 2009 genannt wurde.

Ihre Anwendbarkeit ist jedoch noch begrenzt, da Vergleiche in der Regel nicht in die Menge der zu verwendenden Operationen aufgenommen werden können. Darüber hinaus stellt sich die Frage, ob eine homomorphe Verschlüsselung entwickelt werden kann, die trotz einer großen Anzahl von Operationen ihre Sicherheit aufrechterhält. Diese Schlussfolgerung führt zu einer weiteren Frage, die in Abschnitt 4, „Anwendungsfälle und Vorteile der homomorphen Verschlüsselung“, behandelt wird.

3.1 Von der Theorie zur Anwendung: Schlüsselideen in der HE-Mathematik

Für die Homomorphe Verschlüsselung sind 4 Schlüsselkonzepte von entscheidender Bedeutung [9, 10]

1. Ideale Gitter

Ein Gitter in der Mathematik ist eine regelmäßige Anordnung von Punkten im Raum. Ein ideales Gitter ist ein spezielles Gitter, das in einem Ring von algebraischen Zahlen definiert ist. Ein Ring ist eine algebraische Struktur, die Operationen wie Addition, Subtraktion und Multiplikation erlaubt. Algebraische Zahlen sind Zahlen, die die Lösung einer algebraischen Gleichung sind. Im Zusammenhang mit der vollständig homomorphen Verschlüsselung, wie sie von Gentry eingeführt wurde, werden diese idealen Gitter verwendet, um eine Verschlüsselungsfunktion zu definieren. Die Sicherheit der Verschlüsselung basiert auf der Schwierigkeit, ein spezifisches Problem in Bezug auf diese Gitter zu lösen, welches als Shortest Vector Problem bekannt ist.

– Beispiel

Betrachten wir die Gleichung $x^2 - 2 = 0$.

Die Lösungen dieser Gleichung sind $L = \{\sqrt{2}, \sqrt{-2}\}$.

Diese beiden Zahlen bilden einen Ring von algebraischen Zahlen, da sie addiert, subtrahiert und multipliziert werden können und das Ergebnis immer eine Zahl der Form

$$a + b\sqrt{2}$$

ist, wobei $a, b \in \mathbb{Z}$

(a) Addition: Nehmen wir die Zahlen: $3 + 2\sqrt{2}$ und $1 + \sqrt{2}$.

$$(3 + 2\sqrt{2}) + (1 + \sqrt{2}) \quad (1)$$

$$= 3 + 1 + 2\sqrt{2} + \sqrt{2} \quad (2)$$

$$= 4 + 3\sqrt{2} \quad (3)$$

Das Ergebnis ist weiterhin in der Form $a + b\sqrt{2}$.

(b) Subtraktion: Für die Subtraktion nehmen wir nun $\sqrt{-2}$, daher arbeiten wir mit komplexen Zahlen.

Nehmen wir die Zahlen: $4 + 3\sqrt{-2}$ und $1 + \sqrt{-2}$.

$$(4 + 3\sqrt{-2}) - (1 + \sqrt{-2}) \quad (4)$$

$$= 4 - 1 + 3\sqrt{-2} - \sqrt{-2} \quad (5)$$

$$= 3 + 2\sqrt{-2} \quad (6)$$

$$= 3 + 2i\sqrt{2} \quad (7)$$

Das Ergebnis ist weiterhin in der Form $a + b\sqrt{2}$.

(c) Multiplikation

Es wird wieder mit $\sqrt{-2}$ gerechnet.

Nehmen wir die Zahlen: $4 + 3\sqrt{-2}$ und $1 + \sqrt{-2}$.

$$(2 + 2\sqrt{-2}) \times (1 + 2\sqrt{-2}) \quad (8)$$

$$= 2 + 4\sqrt{-2} + 2\sqrt{-2} + (2\sqrt{-2} \times 2\sqrt{-2}) \quad (9)$$

$$= 2 + 4i\sqrt{2} + 2i\sqrt{2} + (2i\sqrt{2} \times 2i\sqrt{2}) \quad (10)$$

$$= 2 + 4i\sqrt{2} + 2i\sqrt{2} + 4i^2 \times 2 \quad (11)$$

$$= 2 + 4i\sqrt{2} + 2i\sqrt{2} + 4(-1) \times 2 \quad (12)$$

$$= 2 + 4i\sqrt{2} + 2i\sqrt{2} - 8 \quad (13)$$

$$= -6 + 6i\sqrt{2} \quad (14)$$

Das Ergebnis ist weiterhin in der Form $a + b\sqrt{2}$.

Ein ideales Gitter in diesem Ring ist die Menge aller solchen Zahlen. Jede Zahl in diesem Gitter kann als ein Punkt in einem zweidimensionalen Raum dargestellt werden, wobei a die x -Koordinate und b die y -Koordinate ist.

Es ist wichtig zu beachten, dass nicht jede Zahl in diesem Gitter eine Lösung der Gleichung $x^2 - 2 = 0$ ist, sondern nur $L = \{\sqrt{2}, \sqrt{-2}\}$.

2. Ring-LWE-Problem

Das Ring Learning With Errors (Ring LWE)-Problem ist eine Variante des Learning With Errors (LWE)-Problems, das in der Kryptographie Anwendung findet. Es handelt sich hierbei um ein mathematisches Problem, welches aufgrund seiner vermuteten Schwierigkeit zur Konstruktion sicherer kryptographischer Systeme verwendet wird.

Um das Ring LWE-Problem zu verstehen, muss zuerst das LWE-Problem begriffen werden. Das LWE-Problem besteht darin, ein lineares Gleichungssystem zu lösen, in dem die Gleichungen ein gewisses Rauschen oder Fehler enthalten. Das heißt, selbst wenn die Gleichungen bekannt sind, ist die Lösung schwierig, da sie nicht exakt sind.

Beim Ring-LWE-Problem handelt es sich um eine Erweiterung des LWE-Problems, bei dem die Gleichungen in einem mathematischen Objekt namens „Ring“ auftreten. Ein Ring ist eine Menge von Elementen, bei denen Operationen wie Addition und Multiplikation definiert sind. In diesem Fall handelt es sich um Polynomringe als Elemente, da diese „Härter“ zu lösen sind und zugleich effizienter Operationen durchführen können.

– Beispiel

Betrachten wir einen Ring von Polynomen, deren Koeffizienten aus der Menge $M = \{0, 1\}$ stammen und deren Grad < 3 ist. Ein Beispiel für ein Element dieses Rings ist das Polynom $x^2 + 1$.

Nehmen wir an, wir haben ein „Geheimnis“ s im Ring: $s = x + 1$.

Nun wählen wir ein zufälliges Element a aus dem Ring: $a = x^2$.

Das Produkt von a und s wird dann durch einen kleinen "Fehler" e modifiziert, um b zu erhalten. In diesem Beispiel sei $e = 1$, was zu

$$b = x^3 + x^2 + 1$$

führt.

Das Kernproblem des Ring-LWE besteht darin, das Geheimnis s zu ermitteln, wenn nur a, b und die Tatsache bekannt sind, dass e ein „kleiner“ Fehler ist.

Damit erhalten wir:

$$a = x^2 \tag{15}$$

$$b = a \cdot s + e \tag{16}$$

$$e = 1 \tag{17}$$

Das Finden von s aus den gegebenen Werten a, b ist eine Herausforderung, selbst wenn bekannt ist, dass e ein kleiner Fehler ist.

3. Homomorphie

Eine Funktion wird als Homomorph bezeichnet, wenn diese die algebraische Struktur beibehält.

In der Mathematik ist eine algebraische Struktur eine Menge von Elementen und eine Menge von Operationen, die auf diesen Elementen definiert sind. Diese Operationen folgen bestimmten Regeln und Eigenschaften, die die Struktur definieren. Ein einfaches Beispiel für einen Homomorphismus wäre eine Funktion, die die Addition natürlicher Zahlen erhält. Wenn wir zwei natürliche Zahlen addieren und das Ergebnis um 2 erhöhen, erhalten wir eine ganze Zahl. Der Homomorphismus erhält also die algebraische Struktur der Addition zwischen den beiden Mengen.

– Beispiel

Sei gegeben: $\mathbb{N} = \{0, 1, 2, 3, \dots\}$, $\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$.

Die Addition $+$ ist die Operation, die auf diesen Mengen definiert ist.

Die Addition hat folgende Eigenschaften:

- **Abgeschlossenheit:**

$$\forall a, b \in \mathbb{N} : a + b \in \mathbb{N}$$

$$2 + 3 = 5$$

- **Assoziativität:**

$$\forall a, b, c \in \mathbb{N} : (a + b) + c = a + (b + c)$$

$$(2 + 3) + 4 = 9 \text{ und } 2 + (3 + 4) = 9$$

- **neutrales Element:**

$$\exists! e \in \mathbb{N} \forall a \in \mathbb{N} : a + e = a$$

$$a + 0 = a$$

Diese Eigenschaften der Addition machen sie zu einem wichtigen Teil der algebraischen Struktur der natürlichen Zahlen.

Ein Homomorphismus zwischen zwei algebraischen Strukturen ist eine Abbildung, die die Struktur beibehält. Das bedeutet, wenn es einen Homomorphismus $f : \mathbb{N} \rightarrow \mathbb{Z}$ gibt, dann gilt:

$$\forall a, b \in \mathbb{N} : f(a + b) = f(a) + f(b)$$

Ein einfaches Beispiel für einen solchen Homomorphismus ist die Identitätsfunktion id , bei der jede Zahl in $\mathbb{N}$ auf sich selbst in $\mathbb{Z}$ abgebildet wird.

4. Bootstrapping

Das Konzept des Bootstrappings in der vollständig homomorphen Verschlüsselung (FHE) wurde von Craig Gentry in seiner bahnbrechenden Arbeit [9] eingeführt.

Homomorphe Verschlüsselung erlaubt es, Berechnungen auf verschlüsselten Daten durchzuführen, ohne diese entschlüsseln zu müssen. Trotzdem gibt es

ein inhärentes Problem: Jede Operation, die auf den verschlüsselten Daten durchgeführt wird, fügt ein wenig Rauschen hinzu. Wenn zu viele Operationen durchgeführt werden, ist das Rauschen oft so groß, dass die Daten am Ende nicht mehr korrekt entschlüsselt werden können.

Um dieses Problem zu lösen, wurde das Konzept des sogenannten *Bootstrapping* eingeführt.

Durch diesen Prozess wird das Rauschen in den verschlüsselten Daten reduziert, ohne dass die Daten selbst entschlüsselt werden müssen. Es handelt sich bei FHE-Systemen im Wesentlichen um eine Art der „Verschlüsselungs-verschlüsselung“, welche das Rauschen reduziert und somit die „Lebensdauer“ der verschlüsselten Daten verlängert.

Dabei weisen verschlüsselte Daten eine bestimmte „Tiefe“ auf, die im Wesentlichen die Grenze darstellt, wie viele Berechnungen auf den Daten vorgenommen werden können, bevor sie ungenau und somit nutzlos werden.

Jede Berechnung erhöht die Tiefe der Daten.

Bootstrapping kann die Komplexität einer verschlüsselten Datenmenge reduzieren, ohne direkt auf die Daten selbst zugreifen zu müssen. Dadurch wird es möglich, eine unbegrenzte Anzahl von Berechnungen auf den Daten durchzuführen und ihre Verwendbarkeit zu verlängern.

– Beispiel

- **Verschlüsseln der Verschlüsselung:**

Zuerst verschlüsseln wir die bereits verschlüsselte Zahl c erneut, aber diesmal mit einem anderen, *temporären Schlüssel*. Nennen wir diese doppelt verschlüsselte Zahl c' .

$$Enc_{\text{temp}}(c) = c'$$

- **Homomorphe Berechnung:**

Jetzt führen wir eine spezielle homomorphe Berechnung auf c' durch. Diese Berechnung ist so gestaltet, dass sie das Rauschen in c reduziert, wenn c' entschlüsselt wird. Die Berechnung könnte eine einfache Operation wie die Addition mit einer anderen verschlüsselten Zahl sein.

$$c'' = c' + Enc_{\text{temp}}(x)$$

- **Entschlüsselung mit temporärem Schlüssel:**

Jetzt entschlüsseln wir c'' mit dem temporären Schlüssel. Das Ergebnis ist c , aber mit reduziertem Rauschen.

$$Dec_{\text{temp}}(c'') = c$$

- **Ergebnis:**

Wir haben jetzt eine „klarere“ Version von c , die für weitere homomorphe Berechnungen verwendet werden kann, ohne dass das Rauschen zu groß wird.

4 Paillier Crypto System

Nachdem nun die Grundlagen gelegt sind, soll dieses Beispiel zum besseren Verständnis der homomorphen Verschlüsselung dienen.

Um die Lesbarkeit zu verbessern, wurde dieser Abschnitt an das Ende der Arbeit gestellt.

4.1 Praktische HE

Die Anwendungsmöglichkeiten homomorpher Verschlüsselung sind sehr vielfältig und werden nicht nur im akademischen Kontext diskutiert, sondern finden auch in zahlreichen Industrien Anklang.

Es gibt bereits eine Vielzahl theoretischer Ansätze für mögliche Anwendungsbereiche von Homomorpher Verschlüsselung, jedoch wurden bisher nur wenige Fortschritte in der praktischen Umsetzung erzielt.

- Finanzwesen: Ein Beispiel für eine potenzielle Anwendung wäre die sichere Abwicklung von Finanztransaktionen, wobei durch die Verwendung von verschlüsselten Daten nur der Sender und Empfänger der Transaktion Kenntnis vom Betrag und anderen sensiblen Daten hätten [13].
- Gesundheitswesen: Im Bereich der Medizin und des Gesundheitswesens können vollständige elektronische Patientenakten (ePA) verschlüsselt werden [25, 29].
- E-Government: Eine weitere Idee im Bereich des E-Government ist die anonyme Akkumulation von Wähler*innen. Dadurch bleiben die Wahlen geheim [3].
- Netzwerke: Weitere Ideen existieren im Bereich der Netzwerksicherheit und der Smart Grids. Durch das Aufkommen von intelligenten Stromnetzen können mithilfe von Homomorpher Verschlüsselung (HE) der Energieverbrauch genau analysiert werden, ohne dabei Verbraucherdaten offenzulegen. Eine ähnliche Anwendung ist in der Netzwerksicherheit denkbar: Dabei wird der Netzwerkverkehr überwacht, ohne Inhalte oder andere Daten zu entschlüsseln, und auf mögliche Anomalien geachtet [15, 26].

HE kann immer dann angewendet werden, wenn es sich um zu schützende Daten handelt, die verarbeitet oder geändert werden müssen.

Für den langsamen Fortschritt und das Fehlen realer Anwendungsfälle sind vor allem vier Probleme verantwortlich.

1. Leistungsprobleme:

Homomorphe Verschlüsselung ist rechenintensiv und kann daher langsam sein, insbesondere bei großen Datensätzen. Dies kann zu Verzögerungen bei der Datenverarbeitung führen, was in einigen Anwendungsfällen nicht akzeptabel ist.

Die Größe der Schlüssel kann zwischen 70Mb und 2.3Gb betragen, wobei die Laufzeit dann zwischen 30 Sekunden und 30 Minuten beträgt [10].

2. Komplexität:

Die Implementierung der homomorphen Verschlüsselung ist komplex und erfordert ein hohes Maß an Fachwissen in der Kryptographie. Dies kann es für Organisationen schwierig machen, die Technologie zu implementieren und zu warten.

3. Sicherheitsbedenken:

Obwohl die homomorphe Verschlüsselung in der Theorie sicher ist, können bei der Implementierung Sicherheitslücken auftreten.

Darüber hinaus gibt es Bedenken hinsichtlich der Quantencomputer-Resistenz einiger homomorpher Verschlüsselungsschemata. Sprich ob die nächste Generation der Quantencomputer die Rechenleistung besitzt um die Verschlüsselung zu brechen.

4. Standardisierung:

Es gibt derzeit keine allgemein anerkannten Standards für die homomorphe Verschlüsselung, was die Akzeptanz und Implementierung in der Praxis erschweren kann. Jedoch arbeiten einige namenhafte Firmen, Institutionen, Regierungen und Universitäten daran, eine standardisierte Verschlüsselung einzuführen [22].

2018 wurde bereits die erste Ausarbeitung einer solchen Standardisierung veröffentlicht [2].

5 Machine Learning mit Homomorpher Verschlüsselung

Die Einbindung der homomorphen Verschlüsselung in das maschinelle Lernen verspricht bedeutende Fortschritte bei der datenschutzfreundlichen Entwicklung von KI.

In der Praxis könnte ein auf FHE basierendes maschinelles Lernmodell auf verschlüsselten medizinischen Daten trainiert werden, um Muster zu erkennen und Vorhersagen zu treffen, ohne jemals Zugang zu den tatsächlichen Patientendaten zu haben. Dies könnte beispielsweise zur Diagnose von Krankheiten oder zur Erstellung von Behandlungsplänen genutzt werden, ohne die Privatsphäre des Patienten zu gefährden [29].

Ein weiterer Vorteil der Verwendung von FHE in maschinellen Lernumgebungen ist die Möglichkeit, Modelle in einer verteilten Umgebung zu trainieren. Durch den Einsatz von Techniken wie dem föderierten Lernen können Daten von mehreren Parteien gesammelt und gemeinsam genutzt werden, ohne dass die Daten jemals ihre ursprüngliche Quelle verlassen müssen [8].

In der Praxis bietet die Verwendung von FHE in maschinellen Lernmodellen erhebliche Vorteile, beispielsweise im Gesundheitswesen. Ein FHE-basiertes Modell könnte auf verschlüsselten medizinischen Daten trainiert werden, um Muster zu erkennen und Diagnosen oder Behandlungspläne zu erstellen. Dies geschieht, ohne jemals Zugang zu den tatsächlichen, sensiblen Patientendaten zu haben, wodurch die Privatsphäre des Patienten geschützt wird [29].

Darüber hinaus ermöglicht FHE das Trainieren von Modellen in einer verteilten

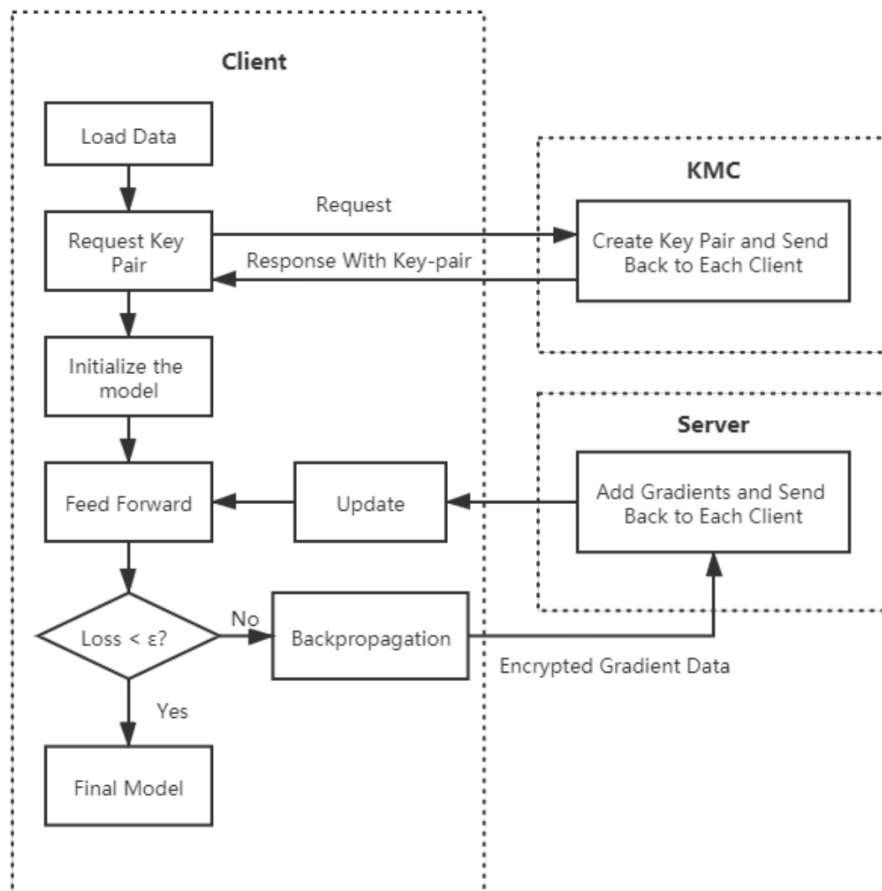


Abbildung 2. Die Architektur als Flowchart eines Paillier Federated Networks [8]

Umgebung durch Techniken wie föderiertes Lernen. Dies erlaubt es, Daten von verschiedenen Quellen zu sammeln und zu nutzen, ohne dass diese ihre ursprüngliche Quelle verlassen müssen, was zusätzlich den Datenschutz erhöht [8].

In Abbildung 2 wird ein Paillier Federated Network dargestellt. Der *Client* wird hier einmal dargestellt, während in der Praxis mehrere *Clients* benutzt werden, um ein verteiltes Netzwerk aufzubauen. Die Clients werden für das Training des Modells genutzt. Sie beginnen damit die Trainingsdaten zu Laden und fordern als nächstes ein Schlüsselpaar an. Das Schlüsselpaar, bestehend aus privat und öffentlichem Schlüssel, werden vom *Key Management Center (KMC)* bereitgestellt. Der Server dient, wie bei anderen Cluster, als Masterknoten.

In einem Paillier Federated Network speichern die einzelnen Knoten ihre Daten

lokal und nur verschlüsselte, aggregierte Informationen werden an den zentralen Server gesendet. Der zentrale Server führt dann Berechnungen mit diesen verschlüsselten Daten durch, um das Modell zu aktualisieren, ohne jemals Zugriff auf die tatsächlichen Daten zu haben. Nach der Aktualisierung wird das Modell wieder an die Knoten verteilt, die es mit ihren eigenen lokalen Daten weiter trainieren.

Dabei gibt es vier Vorteile von Federated Networks [8]:

- **Effizienz:**
Die Möglichkeit, Berechnungen auf verschlüsselten Daten durchzuführen, verringert die Notwendigkeit, Daten hin und her zu senden, und erhöht die Effizienz des Netzwerks.
- **Skalierbarkeit:**
Die Modelle können an einer großen Anzahl von Knotenpunkten trainiert werden, was eine bessere Verallgemeinerung und Leistung zur Folge hat. Die Anzahl der Clients kann einfach und schnell erhöht werden.
- **Datenschutz:**
Die Daten verlassen nie den lokalen Knoten. Alle übertragenen Daten werden verschlüsselt, so dass die Privatsphäre der Benutzer gewahrt bleibt.
- **Compliance:**
Viele Unternehmen haben große Schwierigkeiten, die Datenschutz-Grundverordnung einzuhalten und gleichzeitig einen Mehrwert aus den gewonnenen Daten zu ziehen. Dieser Ansatz kann dazu beitragen, die Einhaltung von Datenschutzbestimmungen wie der DSGVO innerhalb und außerhalb der Europäischen Union zu erleichtern.

5.1 Herausforderung der Implementierung

Trotz des großen Potenzials von FHE im Kontext des maschinellen Lernens gibt es auch erhebliche Herausforderungen.

Eine der wichtigsten ist die Rechenintensität von FHE-Prozessen, die die Trainingszeit von maschinellen Lernmodellen signifikant erhöhen kann.

Darüber hinaus verkompliziert die Integration von Verschlüsselungsmechanismen in das maschinelle Lernen die Modellarchitektur und kann die Verständlichkeit des Modells beeinträchtigen.

Ein weiterer kritischer Punkt ist die Konsistenz der Verschlüsselung. Nur homomorphe Verschlüsselungssysteme bieten die notwendige Konsistenz, um maschinelles Lernen auf verschlüsselten Daten effektiv zu ermöglichen. Andere Verschlüsselungssysteme sind für solche Anwendungen ungeeignet, da sie die notwendige Konsistenz zwischen verschlüsselten und unverschlüsselten Berechnungen nicht gewährleisten können.

In diesem Zusammenhang ist zu beachten, dass ein ML-Modell, welches auf homomorph verschlüsselten Daten trainiert wurde, nur Aussagen über die selbe homomorphe Verschlüsselung machen kann. Daten die mit anderen Schlüsseln verschlüsselt wurden, können mit diesem Modell nicht sinnvoll genutzt werden. Die Aussagen des Modells sind dann unbrauchbar.

In diesem Zusammenhang ist das Paillier-Kryptosystem bemerkenswert, da es sowohl additive als auch multiplikative Homomorphie erlaubt. Es ist jedoch nicht vollständig homomorph und daher nur für eine begrenzte Anzahl von Operationen geeignet. Dies macht es ideal für spezialisierte Anwendungen wie sichere Mehrparteienberechnungen oder elektronische Wahlen, aber weniger praktikabel für komplexere Anwendungen wie maschinelles Lernen. Durch Techniken wie Bootstrapping können die Anwendungsmöglichkeiten des Paillier-Kryptosystems jedoch erweitert werden.

Allerdings gibt es bereits einige öffentlich verfügbare FHE Implementierungen.

1. HELib [11]: Von Halevi und Shoup
2. SEAL [5]: Von Chen et al. und Microsoft
3. TFHE [7]: Von Chillotti et al.

Dies ist nur eine kleine Auswahl der öffentlich Verfügbaren Implementierungen. Alle der oben genannten Beispiele wurden in C++ geschrieben.

5.2 Grenzen und Hürden: HE und ML

In der Praxis hat die homomorphe Verschlüsselung noch keine wirkliche Anwendung gefunden.

Es gibt jedoch einige theoretische Anwendungsfälle. Dazu gehören medizinische, finanzielle und administrative Aufgaben [3]. Auch Data Mining als Ganzes kann mit Hilfe von HE die gleichen Ergebnisse erzielen, jedoch ohne den Verlust der Datensicherheit. Allerdings sind hier noch keine nennenswerten Fortschritte zu verzeichnen.

Ein weiteres Problem, wenn auch nicht so gravierend wie die oben genannten, ist die fehlende Unterstützung für mehrere Benutzer. Für den Fall, dass mehrere Benutzer das gleiche System verwenden, gibt es bisher keine praktikable Lösung. Bisher müsste für jeden Benutzer eine eigene Datenbank angelegt werden. Bei 100 Benutzern wären dies also 100 Datenbanken, was schon bei dieser kleinen Anzahl unpraktisch und aufwändig ist.

López-Alt et. al konnten in ihrem Paper [16] bereits ein Multi-Key FHE Schema darstellen. Dabei können Nutzer unabhängig voneinander Daten in die Cloud senden und empfangen. Nur bei der Entschlüsselung müssen alle beteiligten Nutzer, also alle deren Daten verwendet wurden, mit ihrem Schlüssel zur Entschlüsselung beitragen.

Weiterhin besteht das bereits oben unter Komplexität angesprochene Problem. Bei komplexen, rechenintensiven Anwendungen sind die Laufzeiten für einen praktikablen Einsatz zu lang.

Zusammenfassend lässt sich sagen, dass die vollständig homomorphe Verschlüsselung trotz ihrer vielversprechenden Eigenschaften noch erhebliche Herausforderungen und Einschränkungen aufweist, die ihren praktischen Einsatz erschweren.

6 Zusammenfassung und Ausblick

1. Auswirkungen auf die Datensicherheit

Homomorphe Verschlüsselung revolutioniert die Datensicherheit, insbesondere in Schlüsselbereichen wie Cloud Computing, Big Data und dem Internet der Dinge (IoT). Sie ermöglicht es, Berechnungen auf verschlüsselten Daten durchzuführen, ohne diese entschlüsseln zu müssen, was die Sicherheit erheblich erhöht [4, 6, 14]. Trotz dieser Vorteile bringt die Technologie auch Herausforderungen mit sich, darunter die komplexen Anforderungen an die Implementierung und die zusätzliche Rechenlast, die sie verursacht [18, 23].

2. Zukünftige Möglichkeiten

- Im Bereich des Cloud Computing ermöglicht die homomorphe Verschlüsselung die sichere Speicherung und Verarbeitung von Daten, ohne dass der Cloud-Anbieter Zugriff auf die Daten selbst hat. Dies stellt einen bedeutenden Fortschritt dar, da herkömmliche Verschlüsselungsverfahren zwar die Sicherheit der Daten während der Speicherung und Übertragung gewährleisten, nicht aber während der Verarbeitung [19].
- Durch den Einsatz von homomorpher Verschlüsselung kann die Sicherheit bei der Fernsteuerung und Datenübertragung von intelligenten IoT-Geräten, die auf Cloud-Technologien basieren, deutlich erhöht werden [15].
- Beim Data Mining in der Cloud kann homomorphe Verschlüsselung die nötige Sicherheit bieten und gleichzeitig den Datenschutz auf einem bisher unerreichten Niveau halten.

Damit können private Daten an unbekannte Personen, Firmen etc. gesendet werden, um das eigene ML-Modell zu trainieren oder um die Daten zu analysieren.

7 Anhang

Wie in Kapitel 4 erwähnt, wird hier das Paillier Crypto System dargestellt. Der Einfachheit halber wird hier nur die Addition ausführlich gezeigt. Es ist ebenfalls die Multiplikation möglich, sowie das Verschlüsseln von Texten.

1. Primzahlen:

Wie fast alle Verschlüsselungssysteme basiert auch dieses auf Primzahlen und der Primzahlzerlegung /-faktorisierung.

Wir benötigen zwei ausreichend große Primzahlen mit der gleichen Bitlänge, also der Anzahl der Bits, wenn die Primzahl in Bits dargestellt wird.

Für eine sichere Verschlüsselung sollte die Länge der Primzahlen 1024 bzw. 2048 Bit betragen.

Je länger, desto sicherer. Je länger, desto rechenintensiver ist jedoch die Berechnung der multiplikativen Gruppen.

```

1 from sympy import randprime
2
3 # creating pseudo random prime numbers
4 def create_primes(bits):
5
6     # declare bit-length of primes
7     bit_length = bits
8
9     # Generate first bit-length prime number
10    p = randprime(2**(bit_length-1), 2**bit_length - 1)
11
12    # Generate second bit-length prime number different
13    # from the first one
14    q = randprime(2**(bit_length-1), 2**bit_length - 1)
15    while q == p:
16        q = randprime(2**(bit_length-1), 2**bit_length - 1)
17
18    bit_length_p = p.bit_length()
19    bit_length_q = q.bit_length()
20
21    print(f"Bit length of {p} is: {bit_length_p} bits")
22    print(f"Bit length of {q} is: {bit_length_q} bits")
23    return p, q
24
25 p, q = create_primes(2048)

```

2. größter gemeinsamer Teiler:

Die gewählten Primzahlen müssen:

$$ggT(pq, (p-1) * (q-1)) = 1$$

erfüllen.

Wobei, $n = p \cdot q$ ist und
 $\varphi = (p - 1) \cdot (q - 1)$ die Eulersche Phi Funktion ist.

```

1 n = (p * q)
2 phi = (p - 1) * (q - 1)
3
4 def ggT(a, b):
5     while b != 0:
6         a, b = b, a % b
7     return a
8
9 result_ggT = ggT(n, phi)
10 print("ggT = ", result_ggT)
11 assert result_ggT == 1

```

3. Key Generation:

Public Key:

Der *public Key* besteht aus: n, g .

$n = (p \cdot q)$

g : zufällig gewähltes g aus $(\mathbb{Z}/n^2\mathbb{Z})^*$, so dass n ($n = p \cdot q$) die Ordnung von g teilt.

```

1 import random
2 from sympy import isprime
3
4 # num has to be prime and ggT(num, n^2) == 1
5 def multi_group_g(n):
6     group_g = []
7     for num in range(1, n*n):
8         if isprime(num) and ggT(num, n*n) == 1:
9             group_g.append(num)
10    return group_g
11
12 # random choice of g out of group_g
13 group_g = multi_group_g(n)
14 g = random.choice(group_g)
15
16 print("Multiplicative group(Z/n^2Z)* for n =", n,
17       "contains", len(group_g), "elements.\n Random int g: ", g)
18
19 # final public Key
20 pubKey = n, g
21 print("Public Key: ", pubKey)

```

Private Key:

Der *private Key* besteht aus λ und μ .

– λ

λ ist das kleinste gemeinsame Vielfache (lowest common multiple)

$$\lambda = \text{kgV}(p-1, q-1)$$

$$\text{kgV} = \frac{|a \cdot b|}{\text{ggT}(a, b)}$$

```

1 # kgV
2 def lcm(a, b):
3     return abs(a * b) // ggT(a, b)
4
5 # lambda
6 def calc_lambda(p, q):
7     lambda = lcm(p-1, q-1)
8     return lambda
9
10 lambda = calc_lambda(p, q)
11 print("Lambda: ", lambda)

```

– μ

$$L(x) = \frac{(x-1)}{n}$$

$$\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$$

```

1 # L(x)
2 def L(x, n):
3     return ((x - 1) // n)
4
5 # Mu
6 def calc_mu(g, lambda, n):
7     x = pow(g, lambda, n**2)
8     temp = L(x, n)
9     mu = int(pow(temp, -1, n))
10    return mu
11
12 mu = calc_mu(g, lambda, n)
13
14 # final private Key
15 privKey = lambda, mu
16 print("Private Key: ", privKey)

```

4. Verschlüsselungsprozess:

Wir setzen $m = \text{Klartext}$ mit $m \in (\mathbb{Z}/n\mathbb{Z})$, d.h. $m < n$.

Damit ist der Ciphertext c :

$$c = g^m \cdot r^n \mod n^2$$

r : zufällig gewähltes $r \in (\mathbb{Z}/n\mathbb{Z})^*$

```

1  # num has to be prime and ggT(num, n) == 1
2  def multi_group_r(n):
3      group_r = []
4      for num in range(1, n):
5          if isprime(num) and ggT(num, n) == 1:
6              group_r.append(num)
7      return group_r
8
9  # random choice of r out of group_r
10 def choose_r(n):
11     group_r = multi_group_r(n)
12     r = random.choice(group_r)
13     print("Multiplicative group (Z/nZ)* for n =", n,
14           "contains", len(group_r), "elements.\n Random int
15           r: ", r)
16     return r
17
18 r = choose_r(n)

```

5. Verschlüsselung:

Für die Verschlüsselung benötigen wir den *Public Key* , r und einen Klartext.

```

1  def encryption(g, m, r, n):
2      c = pow(g, m) * pow(r, n) % n**2
3      return c
4
5  # Example Numbers
6  num1 = 2
7  num2 = 7
8
9  c1 = encryption(g, num1, r, n)
10 c2 = encryption(g, num2, r, n)
11
12 print("plaintext num1: ", num1, "\n Ciphertext c1: ", c1
13       , "\n")
14 print("plaintext num2: ", num2, "\n Ciphertext c2: ", c2)

```


6. Entschlüsselungsprozess:

$$m = L(c^\lambda \bmod n^2) \cdot \mu \bmod n$$

Hierbei wird der *Private Key* verwendet.

$L(x)$ ist die oben definierte Funktion.

```

1 def decryption(c, lmbda, n, mu):
2     temp_Lx = pow(c, lmbda, n**2)
3     k = (L(temp_Lx, n) * mu) % n
4     return k
5
6 k1 = decryption(c1, lmbda, n, mu)
7 k2 = decryption(c2, lmbda, n, mu)
8 print("decrypted plaintext num1: ", k1)
9 print("decrypted plaintext num2: ", k2)

```

7. Addition:

Durch Multiplikation zweier Ciphertexte, werden die Klartexte addiert.

$$addCipher = (c_1 \cdot c_2) \bmod n^2$$

```

1 # addition
2 def add_encrypted_numbers(c1, c2, n_square):
3     return (c1 * c2) % n_square
4
5 # added cipher
6 added_cipher = add_encrypted_numbers(c1, c2, n**2)
7 print(added_cipher)
8
9 # added clear text
10 added_decrypted = decryption(added_cipher, lmbda, n, mu)
11 print(f"The sum of {num1} + {num2} = {added_decrypted}")

```

Selbständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst und keine anderen Hilfsmittel als die angegebenen verwendet habe.

Insbesondere versichere ich, dass ich alle wörtlichen und sinngemäßen Übernahmen aus anderen Werken als solche kenntlich gemacht habe.

Literatur

1. Acar, A., Aksu, H., Uluagac, A.S., Conti, M.: A survey on homomorphic encryption schemes: Theory and implementation. *ACM Computing Surveys (Csur)* 51(4), 1–35 (2018)
2. Albrecht, M., Chase, M., Chen, H., Ding, J., Goldwasser, S., Gorbunov, S., Halevi, S., Hoffstein, J., Laine, K., Lauter, K., Lokam, S., Micciancio, D., Moody, D., Morrison, T., Sahai, A., Vaikuntanathan, V.: Homomorphic encryption security standard. Tech. rep., HomomorphicEncryption.org, Toronto, Canada (November 2018)
3. Armknecht, F., Boyd, C., Carr, C., Gjsteen, K., Jschke, A., Reuter, C.A., Strand, M.: A guide to fully homomorphic encryption. *Cryptology ePrint Archive* (2015)
4. Chauhan, K.K., Sanger, A.K., Verma, A.: Homomorphic encryption for data security in cloud computing. In: 2015 International Conference on Information Technology (ICIT). pp. 206–209 (2015)
5. Chen, H., Laine, K., Player, R.: Simple encrypted arithmetic library - seal v2.1. In: Brenner, M., Rohloff, K., Bonneau, J., Miller, A., Ryan, P.Y., Teague, V., Bracciali, A., Sala, M., Pintore, F., Jakobsson, M. (eds.) *Financial Cryptography and Data Security*. pp. 3–18. Springer International Publishing, Cham (2017)
6. Chen, J., You, F.: Application of homomorphic encryption in blockchain data security. In: *Proceedings of the 2020 4th International Conference on Electronic Information Technology and Computer Engineering*. p. 205–209. EITCE ’20, Association for Computing Machinery, New York, NY, USA (2021), <https://doi.org/10.1145/3443467.3443754>
7. Chillotti: Fast fully homomorphic encryption: <https://github.com/tfhe/tfhe>. Github (2017), <https://github.com/tfhe/tfhe>
8. Fang, H., Qian, Q.: Privacy preserving machine learning with homomorphic encryption and federated learning. *Future Internet* 13(4), 94 (2021)
9. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*. p. 169–178. STOC ’09, Association for Computing Machinery, New York, NY, USA (2009), <https://doi.org/10.1145/1536414.1536440>
10. Gentry, C., Halevi, S.: Implementing gentry’s fully-homomorphic encryption scheme. In: *Advances in Cryptology–EUROCRYPT 2011: 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Tallinn, Estonia, May 15–19, 2011. *Proceedings* 30. pp. 129–148. Springer (2011)
11. Halevi, S.: Helib: <https://github.com/homenc/helib>. Github (2013), <https://github.com/homenc/HElib>
12. HindustanTimes: ‘data is the new oil, new gold,’ says pm modi in houston. *Hindustan Times* (Sep 2019), <https://www.hindustantimes.com/india-news/data-is-the-new-oil-new-gold-says-pm-modi-in-houston/story-SphHDPQadvF1dJRMXHCKwK.html>
13. Iezzi, M.: Practical privacy-preserving data science with homomorphic encryption: an overview. In: 2020 IEEE International Conference on Big Data (Big Data). pp. 3979–3988. IEEE (2020)
14. Li, B., Micciancio, D.: On the security of homomorphic encryption on approximate numbers. In: *Advances in Cryptology–EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Zagreb, Croatia, October 17–21, 2021, *Proceedings, Part I* 40. pp. 648–677. Springer, Springer (2021)

15. Liu, H.: Homomorphic encryption-based solution for data security in smart furniture. *Highlights in Science, Engineering and Technology* 39, 926–930 (Apr 2023), <https://drpress.org/ojs/index.php/HSET/article/view/6677>
16. López-Alt, A., Tromer, E., Vaikuntanathan, V.: On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*. pp. 1219–1234 (2012)
17. Matturdi, B., Zhou, X., Li, S., Lin, F.: Big data security and privacy: A review. *China Communications* 11(14), 135–145 (2014)
18. Naehrig, M., Lauter, K., Vaikuntanathan, V.: Can homomorphic encryption be practical? In: *Proceedings of the 3rd ACM Workshop on Cloud Computing Security Workshop*. p. 113–124. CCSW '11, Association for Computing Machinery, New York, NY, USA (2011), <https://doi.org/10.1145/2046660.2046682>
19. R., S.K.G.: Homomorphic encryption using enhanced data encryption scheme for cloud security. *International Journal on Recent and Innovation Trends in Computing and Communication* 6(9) (Apr 2018)
20. Rivest, R.L., Adleman, L., Dertouzos, M.L., et al.: On data banks and privacy homomorphisms. *Foundations of secure computation* 4(11), 169–180 (1978)
21. Samonas, S., Coss, D.: The cia strikes back: Redefining confidentiality, integrity and availability in security. *Journal of Information System Security* 10(3) (2014)
22. Standardization, H.E.: <https://homomorphicencryption.org/> (Apr 2023), <https://homomorphicencryption.org/>
23. Takagi, T., Peyrin, T.: *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security*, Hong Kong, China, December 3–7, 2017, *Proceedings, Part II*, vol. 10625. Springer (2017)
24. Tenzer, F.: Volumen der jährlich generierten/replizierten digitalen datenmenge weltweit von 2010 bis 2022 und prognose bis 2027; <https://de.statista.com/statistik/daten/studie/267974/umfrage/prognose-zum-weltweit-generierten-datenvolumen/>. Statista (May 2023), <https://de.statista.com/statistik/daten/studie/267974/umfrage/prognose-zum-weltweit-generierten-datenvolumen/>
25. Verbraucherzentrale: Elektronische patientenakte (epa): Ihre digitale gesundheitsakte: <https://www.verbraucherzentrale.de/wissen/gesundheitspflege/krankenversicherung/elektronische-patientenakte-epa-ihre-digitale-gesundheitsakte-57223>. Online (Aug 2023), <https://www.verbraucherzentrale.de/wissen/gesundheitspflege/krankenversicherung/elektronische-patientenakte-epa-ihre-digitale-gesundheitsakte-57223>
26. Vinuesa, R., Azizpour, H., Leite, I., Balaam, M., Dignum, V., Domisch, S., Felländer, A., Langhans, S.D., Tegmark, M., Fuso Nerini, F.: The role of artificial intelligence in achieving the sustainable development goals. *Nature communications* 11(1), 233 (2020)
27. WebArchive: Gentry wins acm doctoral dissertation award for innovation in encryption technology (Jun 2023), <https://web.archive.org/web/20160109182814/http://www.acm.org/press-room/awards/dd-award-09>
28. Wegener, C., Milde, T., Dolle, W.: *Informationssicherheits-Management*. Springer (2016)
29. Wood, A., Najarian, K., Kahrobaei, D.: Homomorphic encryption for machine learning in medicine and bioinformatics. *ACM Computing Surveys (CSUR)* 53(4), 1–35 (2020)

Alle Links wurden zuletzt am 23.08.2023 geprüft.